

Project Report

On

Computer Networking: Security

(CSE3752)

Secure E-mail communication with proper authentication using RSA and AES



Submitted by:

Amulya Shrivastava
Anurag Pradhan
Amaan Ahmed
Harsh Singh

Regd. No.: 2341019456
Regd. No.: 2341016359
Regd. No.: 2341001055
Regd. No.: 2341019511

B. Tech. **CSIT** 6th Semester (Section 2341-2F3)

INSTITUTE OF TECHNICAL EDUCATION AND RESEARCH
(FACULTY OF ENGINEERING)
SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY),
BHUBANESWAR, ODISHA

DECLARATION

We, the undersigned students of B. Tech. of (CSE) Department hereby declare that we own the full responsibility for the information, results etc. provided in this PROJECT titled “(Secure E-mail communication with proper authentication using RSA and AES)” submitted to Siksha ‘O’ Anusandhan (Deemed to be University), Bhubaneswar for the partial fulfillment of the subject **Computer Networking: Security (CSE 3752)**. We have taken care in all respect to honor the intellectual property right and have acknowledged the contribution of others for using them in academic purpose and further declare that in case of any violation of intellectual property right or copyright we, as the candidate(s), will be fully responsible for the same.

Name1: Amulya Shrivastava

Regd. No.: 2341019456

Name2: Anurag Pradhan

Regd. No.: 2341016359

Name3: Amaan Ahmed

Regd. No.: 2341001055

Name4: Harsh Singh

Regd. No.: 2341019511

Date: 16th May 2026

Place: Bhubaneswar

Abstract

Electronic mail is one of the most widely used communication methods in today's digital world for sharing personal, educational, and business information. However, normal e-mail communication is vulnerable to several security threats such as unauthorized access, message interception, data modification, phishing, and identity theft. Since e-mails travel through public networks, attackers may capture sensitive information if proper security measures are not applied. Therefore, secure e-mail communication has become an important requirement to ensure confidentiality, integrity, authentication, and privacy of information. To achieve this security, cryptographic techniques such as RSA Cryptosystem and Advanced Encryption Standard are commonly used together.

AES is a symmetric key encryption algorithm that uses the same secret key for both encryption and decryption of data. It is highly efficient, fast, and secure, making it suitable for encrypting large e-mail contents and attachments. In secure e-mail systems, the original message is converted into unreadable cipher text using AES encryption, preventing unauthorized users from accessing the information. However, securely sharing the AES secret key between the sender and receiver is difficult in symmetric encryption systems. To solve this problem, RSA encryption is used.

RSA is an asymmetric encryption algorithm that uses two different keys: a public key and a private key. The sender encrypts the AES secret key using the receiver's RSA public key, and only the receiver can decrypt it using their private key. This ensures secure key exchange over insecure networks. RSA is also used for digital signatures, which help verify the sender's identity and maintain message integrity. The sender creates a digital signature using their private key, and the receiver verifies it using the sender's public key. This process ensures that the message has not been altered during transmission and protects against impersonation attacks.

The combination of RSA and AES creates a hybrid cryptographic system that provides both strong security and high performance. AES offers fast encryption for large amounts of data, while RSA provides secure key management and authentication. Together, these algorithms ensure secure and reliable e-mail communication. This hybrid approach is widely used in modern secure e-mail protocols because it effectively protects sensitive information from cyber threats and unauthorized access.

Contents

Serial No.	Chapter No.	Title Of The Chapter	Page No.
1.	1	Introduction	1
2.	2	Problem Statement	2
3.	3	Methodology	3 – 4
4.	4	Implementation	5 – 9
5.	5	Results and Interpretation	10 – 12
6.	6	Conclusion	13
7.		References	14

1. Introduction

Electronic mail, commonly known as e-mail, has become one of the most important and widely used methods of communication in the modern world. Individuals, organizations, educational institutions, and businesses rely heavily on e-mail for exchanging information, documents, and confidential data quickly and efficiently. Although e-mail communication offers convenience and speed, it also faces several security challenges such as unauthorized access, data interception, message tampering, phishing attacks, identity theft, and impersonation. Since e-mails are transmitted over public networks, attackers can capture or modify sensitive information if proper security mechanisms are not implemented. Therefore, ensuring secure e-mail communication with proper authentication has become a major concern in the field of cybersecurity.

To protect e-mail data from cyber threats, encryption techniques are widely used. Encryption converts readable information, known as plain text, into an unreadable format called cipher text, which can only be understood by authorized users. Among the various cryptographic algorithms available, RSA Cryptosystem and Advanced Encryption Standard are the most commonly used algorithms for secure communication systems. These algorithms provide confidentiality, integrity, authentication, and secure key management, making them highly suitable for protecting e-mail communication.

AES is a symmetric key encryption algorithm in which the same secret key is used for both encryption and decryption. It is known for its high speed, efficiency, and strong security. AES supports different key sizes such as 128-bit, 192-bit, and 256-bit keys, providing protection against brute-force attacks and unauthorized access. In secure e-mail communication, AES is mainly used to encrypt the actual message content and attachments because it can process large amounts of data quickly and efficiently. However, securely sharing the AES secret key between the sender and receiver becomes a major challenge in symmetric encryption systems.

To solve the problem of secure key exchange, RSA is used along with AES. RSA is an asymmetric encryption algorithm that uses two keys: a public key and a private key. The public key is shared openly and is used for encryption, while the private key is kept secret and is used for decryption. In a secure e-mail system, the sender encrypts the e-mail using AES and then encrypts the AES secret key using the receiver's RSA public key. Only the receiver can decrypt the secret key using their private key, ensuring secure communication. RSA also supports digital signatures, which provide authentication and message integrity. Digital signatures verify the identity of the sender and ensure that the message has not been altered during transmission.

The combination of RSA and AES forms a hybrid cryptographic system that provides both high security and better performance. AES offers fast encryption for large data, while RSA ensures secure key exchange and authentication. This hybrid approach is widely used in modern secure e-mail protocols such as S/MIME and PGP. Therefore, secure e-mail communication using RSA and AES plays a significant role in protecting sensitive information, maintaining privacy, and ensuring trusted communication in today's digital environment.

2. Problem Statement

Electronic mail has become one of the most important and widely used communication methods in personal, educational, and business environments. People frequently use e-mail services to exchange confidential information, important documents, passwords, and sensitive data. Although e-mail communication provides speed and convenience, traditional e-mail systems are highly vulnerable to security threats such as unauthorized access, message interception, phishing attacks, identity theft, impersonation, and data tampering. Since e-mails are transmitted through public networks, attackers can easily capture or modify the information during transmission, which may lead to loss of privacy, confidentiality, and trust.

One of the major problems in existing e-mail systems is the lack of proper security mechanisms for encryption and authentication. Users cannot always verify whether the received message is actually sent by the claimed sender or whether the message content has been altered during transmission. These security weaknesses increase the risk of cyberattacks and misuse of sensitive information. Therefore, there is a strong need for a secure e-mail communication system that can ensure confidentiality, integrity, authentication, and secure transmission of data.

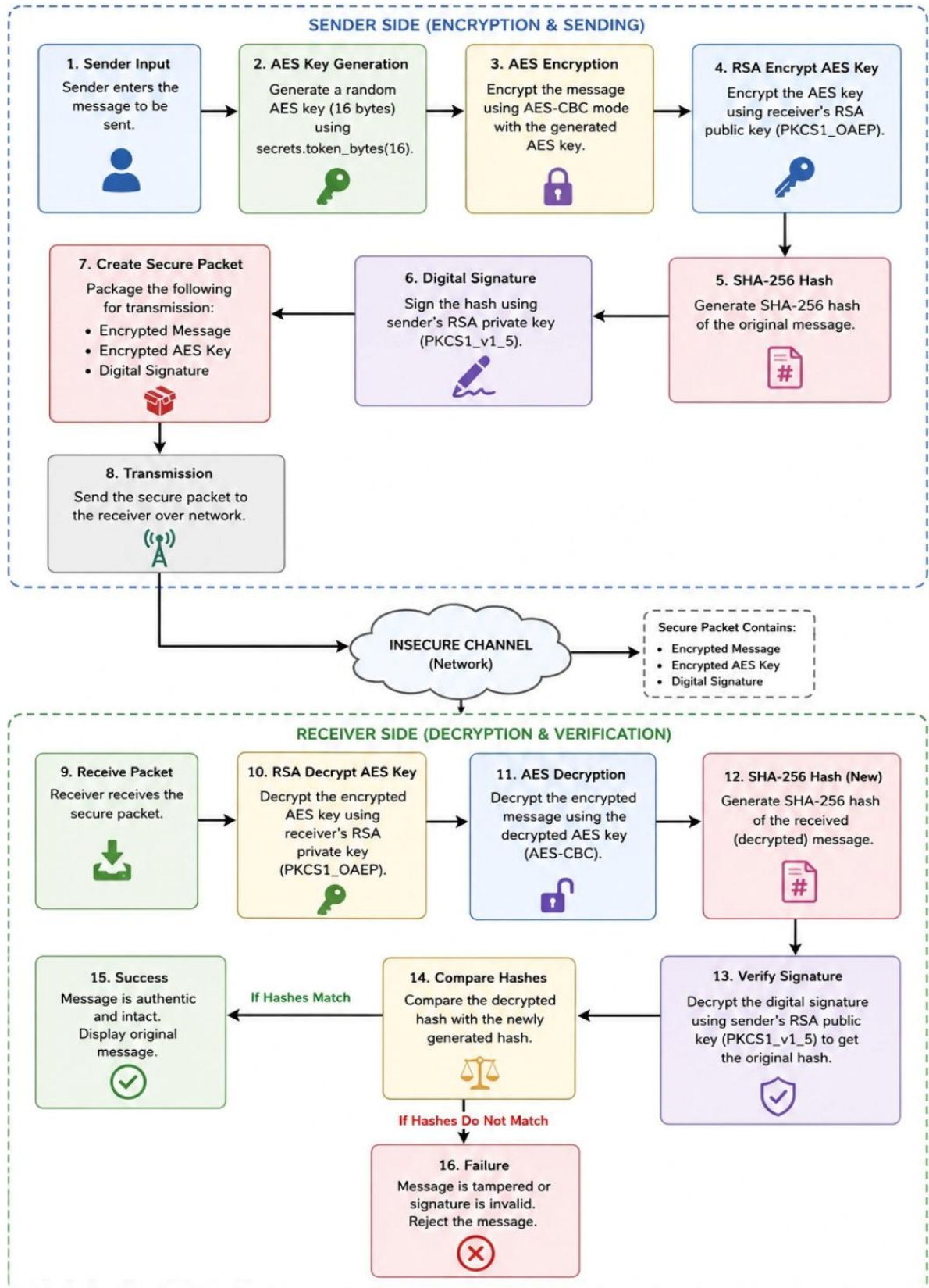
To overcome these issues, this project proposes secure e-mail communication using RSA Cryptosystem and Advanced Encryption Standard. AES is used to encrypt the e-mail content and attachments because it provides fast and efficient encryption for large amounts of data, while RSA is used for secure key exchange and digital signatures to provide authentication and integrity. The main objective of this project is to develop a secure and reliable e-mail communication system that protects sensitive information from unauthorized access and ensures safe communication over insecure networks.

Objectives to fulfill:

- I. To study and understand the AES and RSA algorithms, focusing on their roles in encryption, key management, and authentication.
- II. To implement AES for encrypting and decrypting messages, ensuring the confidentiality of communication.
- III. To use RSA for secure key exchange and digital signatures, enabling authentication and integrity verification.
- IV. To design a mechanism for verifying sender identity using RSA-based digital signatures.
- V. To develop a complete communication system that supports secure, authenticated messaging between users.
- VI. To evaluate the system's effectiveness in terms of encryption speed, security level, and resistance to attacks such as man-in-the-middle or impersonation.

3. Methodology

I. Block diagram of cryptosystem used in project:



II. Algorithm or Pseudocode for implementation of the project.

- **Algorithm: Secure E-mail Communication using RSA and AES**
- Step 1: Start the program.
- Step 2: Generate RSA public and private keys for sender and receiver.
- Step 3: Accept the e-mail message from the sender.
- Step 4: Generate a random AES secret key.
- Step 5: Encrypt the e-mail message using AES encryption.
- Step 6: Encrypt the AES secret key using receiver's RSA public key.
- Step 7: Generate SHA-256 hash value of the original message.
- Step 8: Create digital signature using sender's RSA private key.
- Step 9: Create a secure packet containing:
 - Encrypted message
 - Encrypted AES key
 - Digital signature
 - Nonce and authentication tag
- Step 10: Transmit the secure packet through the network.
- Step 11: Receiver decrypts the AES key using RSA private key.
- Step 12: Receiver decrypts the encrypted message using AES decryption.
- Step 13: Verify the digital signature using sender's RSA public key.
- Step 14: If verification is successful, display the original message.
- Step 15: Otherwise, reject the message and display verification failure.
- Step 16: Stop the program.

III. Tools used for implementation of algorithm/pseudocode.

Sl. No.	Tool / Technology	Purpose
1.	Python	Main programming language used for implementation.
2.	PyCryptodome Library	Used for RSA, AES, SHA-256, and Digital Signature operations.
3.	RSA Algorithm	Used for secure key exchange and authentication.
4.	AES Algorithm	Used for encryption and decryption of e-mail messages.
5.	SHA-256	Used for hashing and integrity verification.
6.	JSON	Used for storing encrypted data and secure packet.
7.	Jupyter Notebook / VS Code	Used for writing and executing Python code.
8.	Operating System	Platform used for execution of the program.

4. Implementation

Install PyCryptodome library

```
!pip install pycryptodome
```

Display project title

```
print("=" * 60)
print("SECURE E-MAIL COMMUNICATION USING RSA AND AES")
print("=" * 60)
```

Import required libraries

```
import os
import json
import base64
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.PublicKey import RSA
from Crypto.Random import get_random_bytes
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
```

Create folders to store keys and encrypted messages

```
os.makedirs("keys", exist_ok=True)
os.makedirs("messages", exist_ok=True)
print("Folders created successfully.")
```

Function to generate RSA public and private keys

```
def generate_keys(user_name):
```

Generate 2048-bit RSA key pair

```
key = RSA.generate(2048)
```

Export private and public keys

```
private_key = key.export_key()
public_key = key.publickey().export_key()
```

Save private key in keys folder

```
with open(f'keys/{user_name}_private.pem', "wb") as f:
    f.write(private_key)
```

Save public key in keys folder

```
with open(f'keys/{user_name}_public.pem', "wb") as f:
    f.write(public_key)
```

```

print(f'{user_name} RSA keys generated.")
# Generate keys for sender and receiver
generate_keys("sender")
generate_keys("receiver")

# Function to load RSA public key
def load_public_key(path):
    with open(path, "rb") as f:
        key = RSA.import_key(f.read())
    return key

# Function to load RSA private key
def load_private_key(path):
    with open(path, "rb") as f:
        key = RSA.import_key(f.read())
    return key

# Function to encrypt message using AES
def aes_encrypt(message):
    # Generate random 256-bit AES key
    aes_key = get_random_bytes(32)

    # Create AES cipher in EAX mode
    cipher = AES.new(aes_key, AES.MODE_EAX)

    # Encrypt message and generate authentication tag
    ciphertext, tag = cipher.encrypt_and_digest(message.encode())

    # Return encrypted data
    return {"aes_key": aes_key, "ciphertext":
        base64.b64encode(ciphertext).decode(), "nonce":
        base64.b64encode(cipher.nonce).decode(), "tag":
        base64.b64encode(tag).decode()}

# Function to decrypt AES encrypted message
def aes_decrypt(aes_key, ciphertext, nonce, tag):

    # Recreate AES cipher using nonce
    cipher=AES.new(aes_key,AES.MODE_EAX,nonce=base64.b64decode(nonce
    ))

    # Decrypt and verify integrity using tag
    decrypted_message = cipher.decrypt_and_verify(
        base64.b64decode(ciphertext),

```

```

base64.b64decode(tag))return decrypted_message.decode()
# Function to encrypt AES key using RSA public key
def rsa_encrypt_key(aes_key, public_key):

# Create RSA cipher object
cipher_rsa = PKCS1_OAEP.new(public_key)

# Encrypt AES key
encrypted_key = cipher_rsa.encrypt(aes_key)
return base64.b64encode(encrypted_key).decode()

# Function to decrypt AES key using RSA private key
def rsa_decrypt_key(encrypted_key, private_key):

# Create RSA cipher object
cipher_rsa = PKCS1_OAEP.new(private_key)

# Decrypt AES key
decrypted_key = cipher_rsa.decrypt(base64.b64decode(encrypted_key))
return decrypted_key

# Function to create digital signature
def sign_message(message, private_key):

# Create SHA-256 hash of message
hash_value = SHA256.new(message.encode())

# Sign hash using sender's private key
signature = pkcs1_15.new(private_key).sign(hash_value)
return base64.b64encode(signature).decode()

# Function to verify digital signature
def verify_signature(message, signature, public_key):

# Create SHA-256 hash of received message
hash_value = SHA256.new(message.encode())
try:
# Verify signature using sender's public key
pkcs1_15.new(public_key).verify(hash_value,base64.b64decode(signature))
return True
except:
return False

# Take e-mail message input from user

```

```

email_message = input("Enter the e-mail message: ")
print("\nEncrypting message using AES...")

# Encrypt the message using AES
encrypted_data = aes_encrypt(email_message)

# Load receiver's public key
receiver_public_key = load_public_key(
    "keys/receiver_public.pem")

# Encrypt AES key using receiver's RSA public key
encrypted_aes_key = rsa_encrypt_key(
    encrypted_data["aes_key"],
    receiver_public_key)
print("AES key encrypted using RSA.")

# Load sender's private key
sender_private_key = load_private_key(
    "keys/sender_private.pem")

# Generate digital signature for original message
signature = sign_message(email_message,sender_private_key)
print("Digital signature generated.")

# Create secure e-mail packet
secure_email_packet = {"encrypted_message": encrypted_data["ciphertext"],
    "nonce": encrypted_data["nonce"],
    "tag": encrypted_data["tag"],
    "encrypted_aes_key": encrypted_aes_key,
    "signature": signature}

# Save secure packet in JSON file
with open("messages/secure_email.json", "w") as f:
    json.dump(secure_email_packet, f, indent=4)
print("\nSecure e-mail packet created successfully.")

# Read secure packet from file
with open("messages/secure_email.json", "r") as f:
    received_packet = json.load(f)
print("Secure e-mail packet received.")

# Load receiver's private key
receiver_private_key = load_private_key(
    "keys/receiver_private.pem")

```

```

# Decrypt AES key using receiver's private key
decrypted_aes_key = rsa_decrypt_key(
received_packet["encrypted_aes_key"],
receiver_private_key)
print("AES key decrypted.")

# Decrypt original message using AES key
decrypted_message = aes_decrypt(
decrypted_aes_key,received_packet["encrypted_message"],
received_packet["nonce"],received_packet["tag"])

# Display decrypted message
print("\nDecrypted Message:")
print(decrypted_message)

# Load sender's public key
sender_public_key = load_public_key(
"keys/sender_public.pem")

# Verify digital signature
is_verified = verify_signature(decrypted_message,received_packet["signature"],
sender_public_key)
print("\nSignature Verification Status:")

# Display verification result
if is_verified:
    print("Signature Verified Successfully.")
else:
    print("Signature Verification Failed.")

# Display completion message
print("\n" + "=" * 60)
print("SECURE COMMUNICATION COMPLETED")
print("=" * 60)

```

5. Results & Interpretation

- 1) The console output showing “sender keys generated successfully” and “receiver keys generated successfully” confirms that the system has created two pairs of RSA keys. The sender’s private key is used for creating digital signatures, while the sender’s public key is used for signature verification. Similarly, the receiver’s public key is used to encrypt the AES secret key, and the receiver’s private key is used to decrypt it securely.

```
PS D:\OneDrive\Desktop\secure_mail_project> python
e\Desktop\secure_mail_project\generate_keys.py"
sender keys generated successfully
receiver keys generated successfully
```

- 2) After key generation, the sender executes the sender.py file and enters a message such as “hello”. Once the message is entered, the system generates a random AES secret key and encrypts the message using the AES algorithm. AES encryption converts the original plain text message into unreadable ciphertext, ensuring confidentiality of the data. Along with encryption, AES also generates a nonce and authentication tag, which help maintain message integrity and prevent tampering. Since AES is a symmetric encryption algorithm, the same AES key will later be required to decrypt the message.

```
PS D:\OneDrive\Desktop\secure_mail_project> python
e\Desktop\secure_mail_project\sender.py"
Enter Message: hello

Message Encrypted Using AES

AES Key Encrypted Using RSA

Encrypted AES Key:
fSutY49xK+dK0FAEhQ3kG9xia74pMc8eLThTx4Xho1Kf58RwcT4
oaFt0JDYYfh5Xbo+y16knLFLwQMy7MLv4q46ew+rkSLhrwLftV
sQbQTu/JGFbhW0lQ6TWT67BNA1rmlwsejc1Vtky04prZzMxa/bD
g2yzqdThKr85S+0sftt/jQmSxSnm3e3yQ4C9xBCZpouB+Xpi4Sc
7JKYq+XFPKcPHLU/7DcSVyQjDhZerFx1Co6931cI1acFk4WPKB3
iImiiUwHMJiCipONA==

Digital Signature Created

Secure Message Sent Successfully
```

- 3) This JSON object represents a securely encrypted message using both AES and RSA encryption techniques. The original message is encrypted using AES, where the ciphertext, nonce, and tag ensure confidentiality and integrity of the message. The AES secret key is securely encrypted using the receiver's RSA public key so that only the receiver can decrypt it. A digital signature is also generated using the sender's RSA private key to verify sender authenticity and prevent message tampering.

```
ges > {} message.json > ...
{
  "original_message": "hello",
  "ciphertext": "/opPMf8=",
  "nonce": "8fzibDMoV7iiRU9obxvCrg==",
  "tag": "SI5zRfIOw9pfQRsUrbDNNw==",
  "encrypted_aes_key": "fSutY49xK
+dK0FAEHQ3kG9xia74pMc8eLThTx4Xho1kf58RWcT4qEV03UPL56a0WpoaFt0JDYYfh5Xbo
+y16knLFLwQMy7MLv4q46eW+rKSLhrwLftVD204ATowjAZWAasQbQTu/
JGFbhw0lQ6TWT67BNA1rmlwsejc1Vtky04prZzMxa/bDgYc3mI0lKB17Dhg2yzqdThKr85S+0sftt/
jQmSxSnm3e3yQ4C9xBCZpouB+Xpi4SqZMq97+qRdAsGxF7JKYq+XFPKcPHLU/
7DcSVyQjDhZerFx1Co6931cI1acFk4WPKBJy8fEYOY7NUoKZ0iImiiUwHmJicipONA==",
  "signature": "OX64HmdQGWBG2gkpMX5pTAW/
BwsbXSMziaZFyvgrPXESHnYHQy6qCJYTHKeHGAPh2phKB1P1
+Gq8MX1yzzVwL2tc6atB4Q9BRmqGksFZaSG/eqbtZIVpVin3Iw0bY3oA5PbUE
+E5w6SlpZ3WErYjhUIU8ziXwGb+7s/uDORxflVewhjc8uBuNJz16Dto/
5d2Ai2Ki6MEkak98rdHnCjyVS4g1MmTSP1XN9yNo5hvEUNsb45Cr6RLz5nF8AIi4q/RCC6d7tE
+i9RZoT9hVRaoHQK2JPITQakr3gUc92FobCLWGcgJY0MwEw0PG6UbluXxGSNSwsFQXa6SElFySyORu
Q=="
}
```

If someone tries to tamper a message it will show

- 4) The AES key was successfully decrypted, allowing the receiver to recover the original message securely. The digital signature verification and message match confirm that the message is authentic, intact, and not tampered with during transmission.

```
● PS D:\OneDrive\Desktop\secure_mail_project> python
project\reciever.py"
AES Key Decrypted Successfully

Message Match Successful

Signature Verified
Message Authentic
```

5.) The error message in your image indicates that the **kernel has crashed** while executing code related to cryptographic operations (AES decryption and signature verification). Based on the output, the crash happened immediately after performing a signature check. If someone tries tamper the message(email) it will show email tamperment due to data authentication and integrity.

```
messages > {} message.json > ** ciphertext
1
2 "original_message": "hello",
3 "ciphertext": "hKP9feU=",
4 "nonce": "+E778L4oesas5WQI64Kwuw==",
5 "tag": "GtqMPz9zXTDWyxanubiYA==",
6 "encrypted_aes_key": "uqhG2b9WQekLWkQ93jyLUKHONTIE/vNdiKtRs42o52j5gNCg5ZCBW1a+3eHfjXemr71UE6bmqs2L4WwXuve1dAf5Z06V6FF5MmpVF-v7FQYdjYoSDcuxTJs3jFQteImLdkr/I+axw8AHQA14vcg
7 "signature": "Db21UmxCVpepqJuTD93NDS+dx1xw41bEgQtS/p8gcbvLclMTI78eU7jEt6L6DjUC5iuChymK8kFRmdPcQ2709L4HzZDSm4DTrmvtv9paZ55TIZfZFPzyPwDtS6zPUQtKSnW0JAT6v6QDBCTxzSdf4guF7BxiU
8
```

Secure Message Packet Received

AES Key Decrypted Successfully

Message Tampered

Signature Verified

Message Authentic

6. Conclusion

The project “Secure E-mail Communication with Proper Authentication using RSA and AES” successfully demonstrates the implementation of a secure and reliable communication system using modern cryptographic techniques. The main objective of the project was to provide confidentiality, authentication, integrity, and secure key exchange during e-mail communication over insecure networks. This objective was achieved by combining the advantages of both symmetric and asymmetric encryption algorithms in a hybrid cryptographic system.

In this implementation, AES (Advanced Encryption Standard) was used to encrypt the e-mail message because of its high speed and strong security features. RSA algorithm was used for secure exchange of the AES secret key and for digital signature generation and verification. SHA-256 hashing algorithm was implemented to verify the integrity of the transmitted message and ensure that the data was not modified during transmission. The project also demonstrated the process of encryption, decryption, digital signing, and authentication in a practical and efficient manner using Python programming language and cryptographic libraries.

The successful execution of the project proved that hybrid encryption techniques provide better security and performance for secure communication systems. The project enhanced the understanding of important concepts of network security such as symmetric encryption, asymmetric encryption, hashing, digital signatures, and secure key management. Overall, this project provides a strong foundation for developing secure communication applications and highlights the importance of cryptography in protecting sensitive information in modern digital communication systems.

7. References

- CompTIA Security+ SY0-701 Certification Guide by Ian Neil, 3rd Edition, Packt publication.
- William Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson Education.
- Atul Kahate, *Cryptography and Network Security*, Tata McGraw Hill.
- National Institute of Standards and Technology (NIST), “Advanced Encryption Standard (AES)”